

Unit I :: Introduction to a Programming Environment

☞ **SEC-01-PHY-91-R-03** ☞

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit I: Introduction to a Programming Environment

Software installation and start-up; locating and launching the application, exploring the graphical interface of the software: console, script editor, variable browser; understanding the workspace: current directory, command history, working directory settings; using built-in help and documentation features; basic command execution: arithmetic expressions, variable assignments, managing output display; data types and variables: numeric types (integers, floats), strings, booleans; variable naming conventions; operators and expressions: arithmetic, relational, logical operators; operator precedence and associativity; code structure and readability: using comments, breaking long expressions, line continuation symbols; saving and executing scripts, running code from the script editor or the command window.

1 Setting up the Scilab software

1.1 Installing Scilab

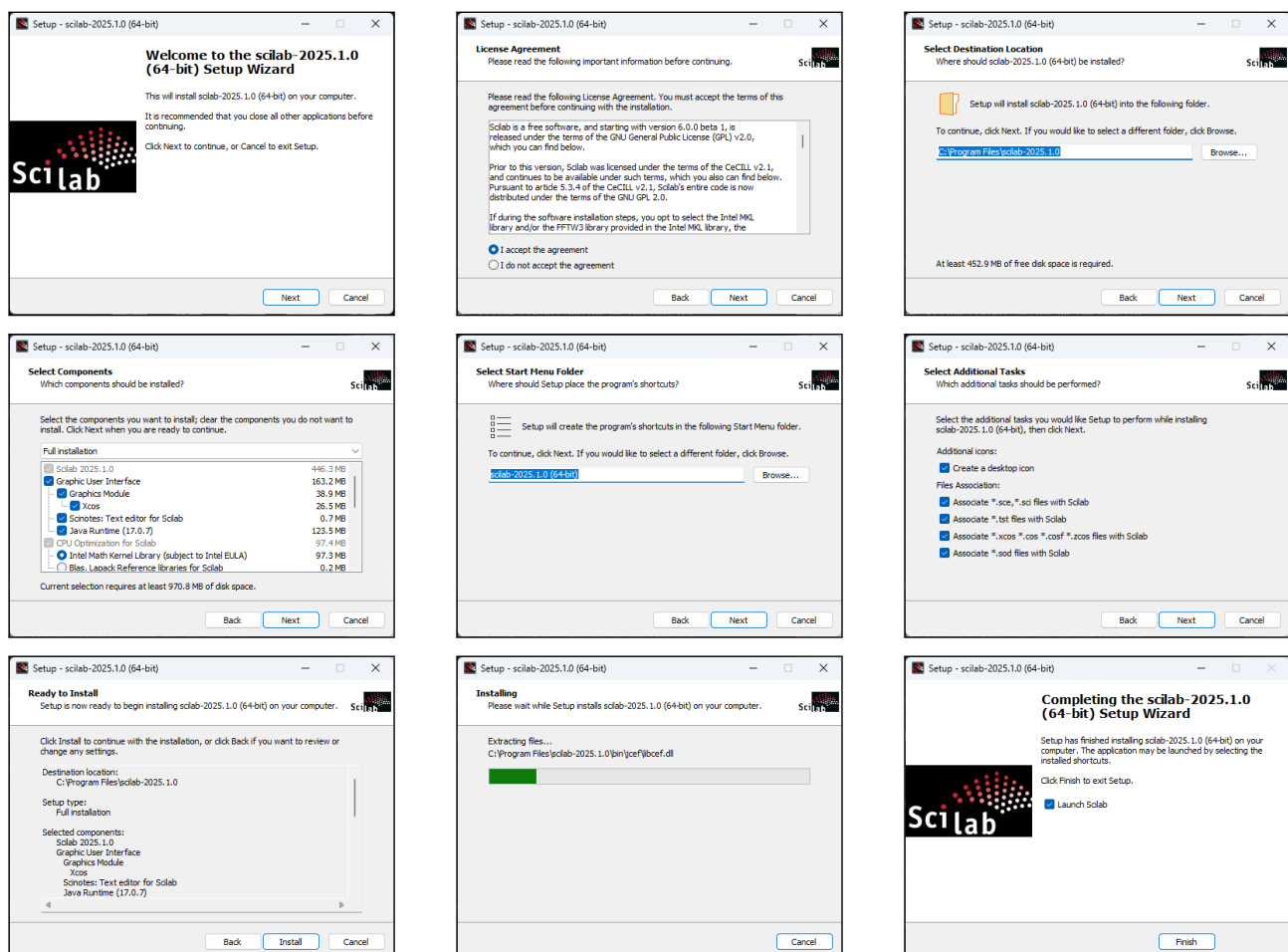
I. Visit the download page of Scilab available at: <https://www.scilab.org/download/>



Figure 1: The webpage for downloading the latest version of Scilab

II. Click on the relevant link to download Scilab for Windows, Linux or MacOS.

III. Navigate to the folder containing the downloaded file and install the software on the PC. Select 'English' as the preferred language the then click on 'OK' to proceed with the rest of the installation process as illustrated below:



1.2 Scilab on Cloud

The *National Virtual Library of India*¹ maintains an online version of Scilab on the Garuda Cloud. This is known as *Scilab on Cloud* and is available at: <https://cloud.scilab.in/>

The online interface of *Scilab on Cloud* is displayed in figure 3.

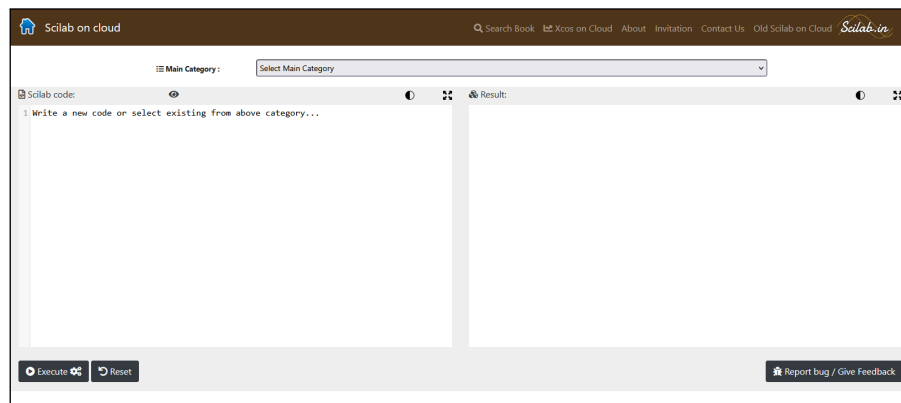


Figure 3: Scilab on Cloud

One can type the Scilab code on the left panel, then click on 'Execute' to view the output on the right panel.

1.3 Launching Scilab

One can find the Scilab icon on the desktop or in the list of installed applications on one's PC. Simply clicking on the icon launches Scilab with its default GUI which is displayed in figure 4 below.

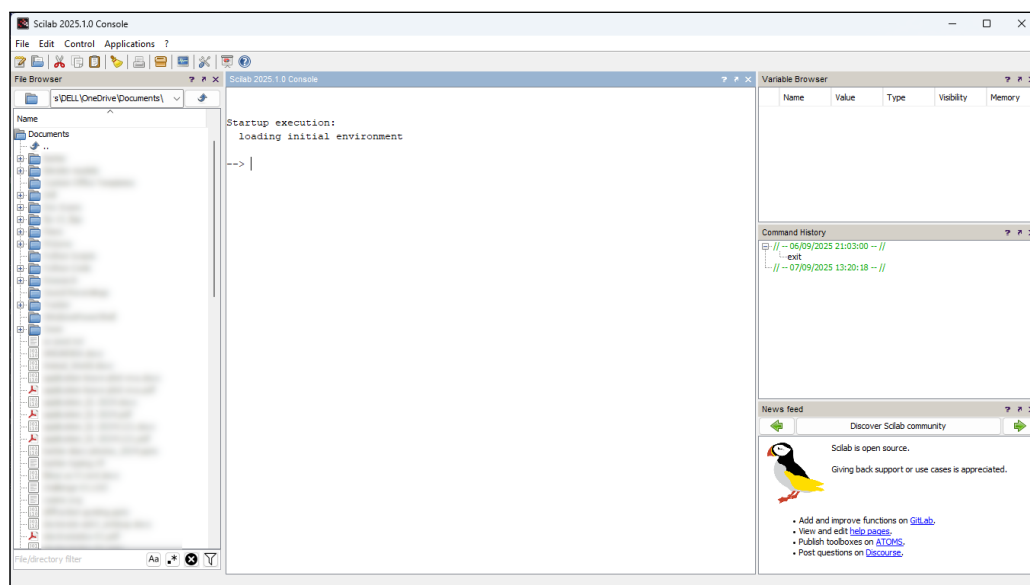


Figure 4: Default graphical user interface (GUI) of Scilab

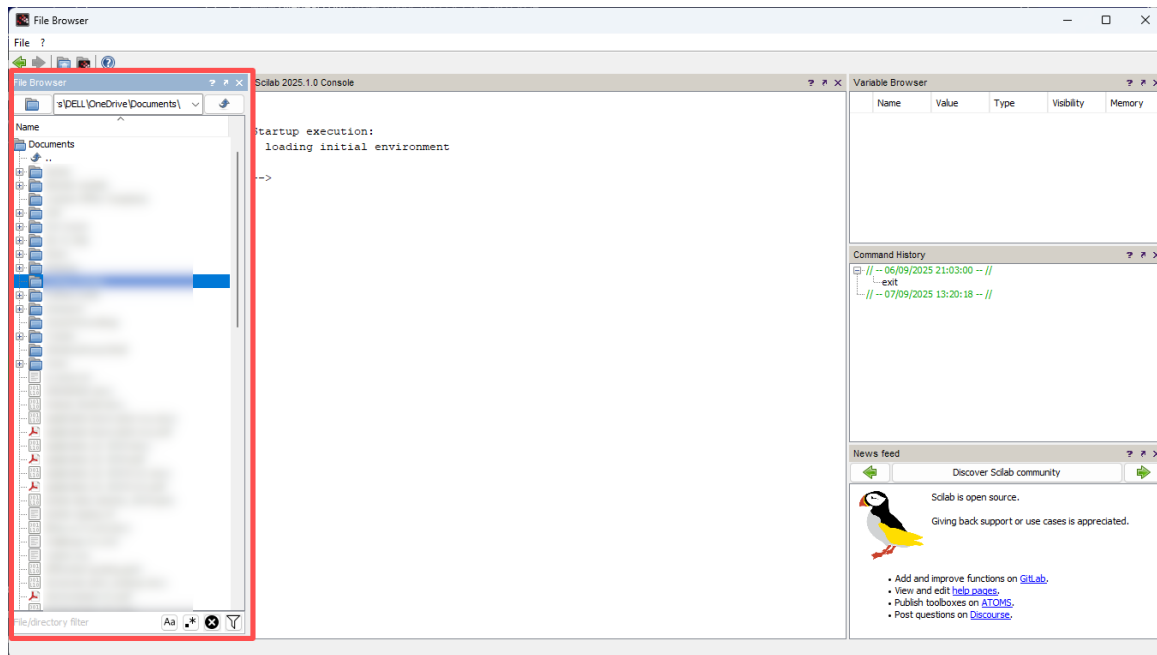
¹<https://www.rrr1f.gov.in/NML/NVLI.aspx>

2 Understanding the GUI of Scilab

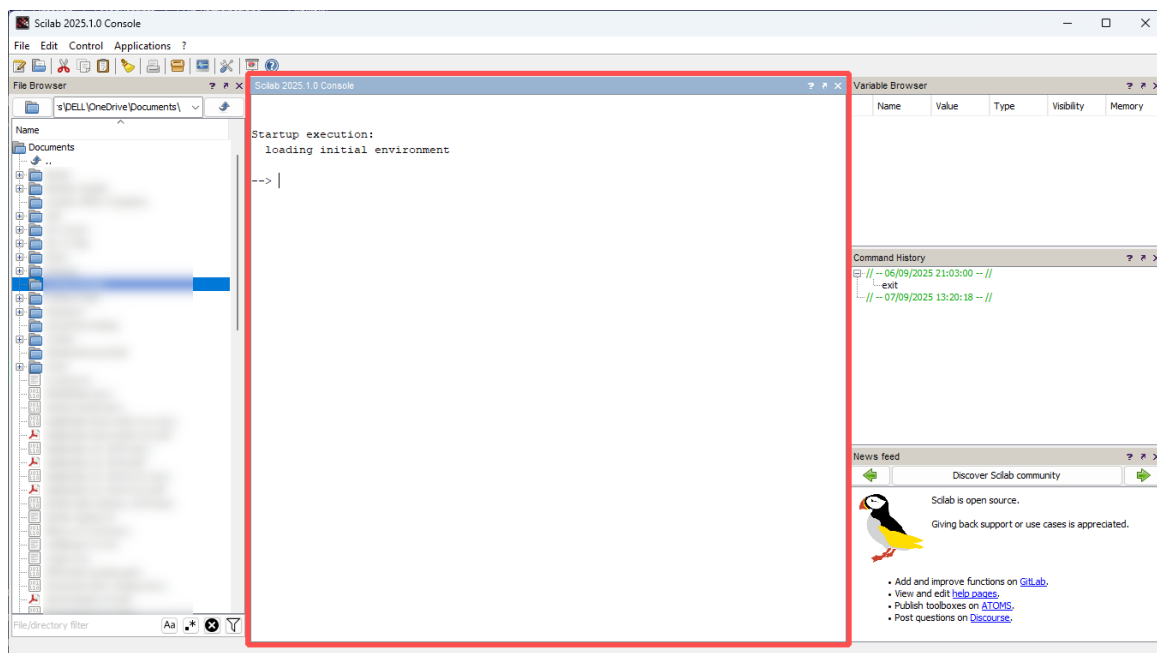
2.1 Panes and layout

The layout of Scilab's GUI is divided into five distinct panes, each of which has its own unique functionality.

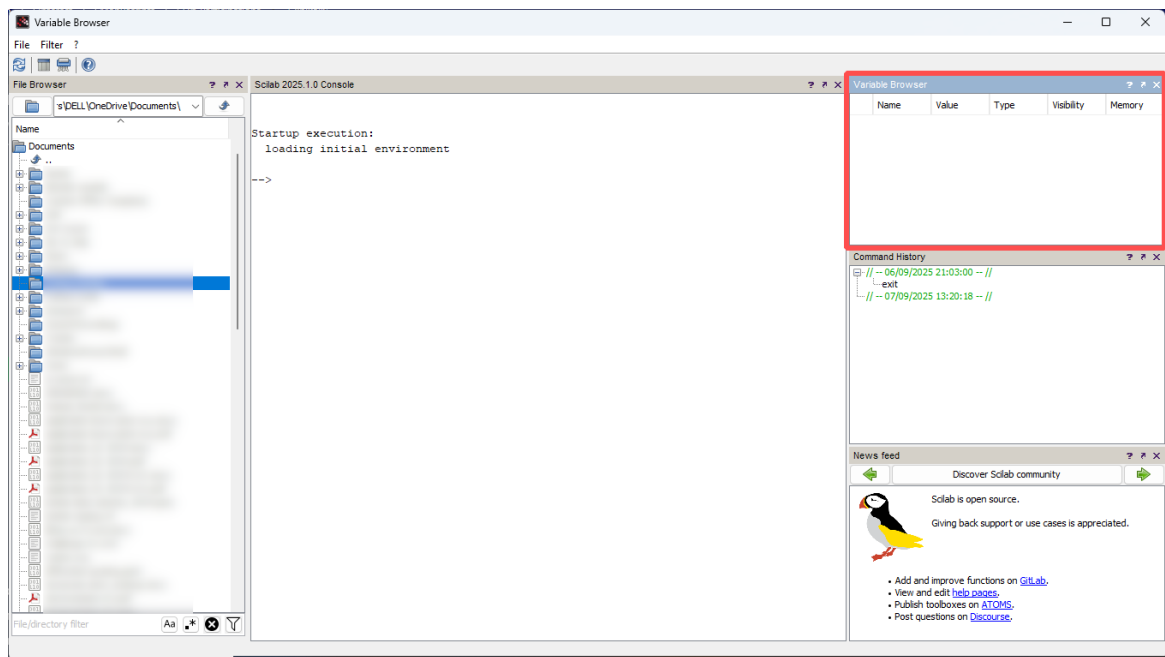
1. *File browser* – allows one to browse through the directory tree to easily navigate to the relevant Scilab script file or folder containing the same.



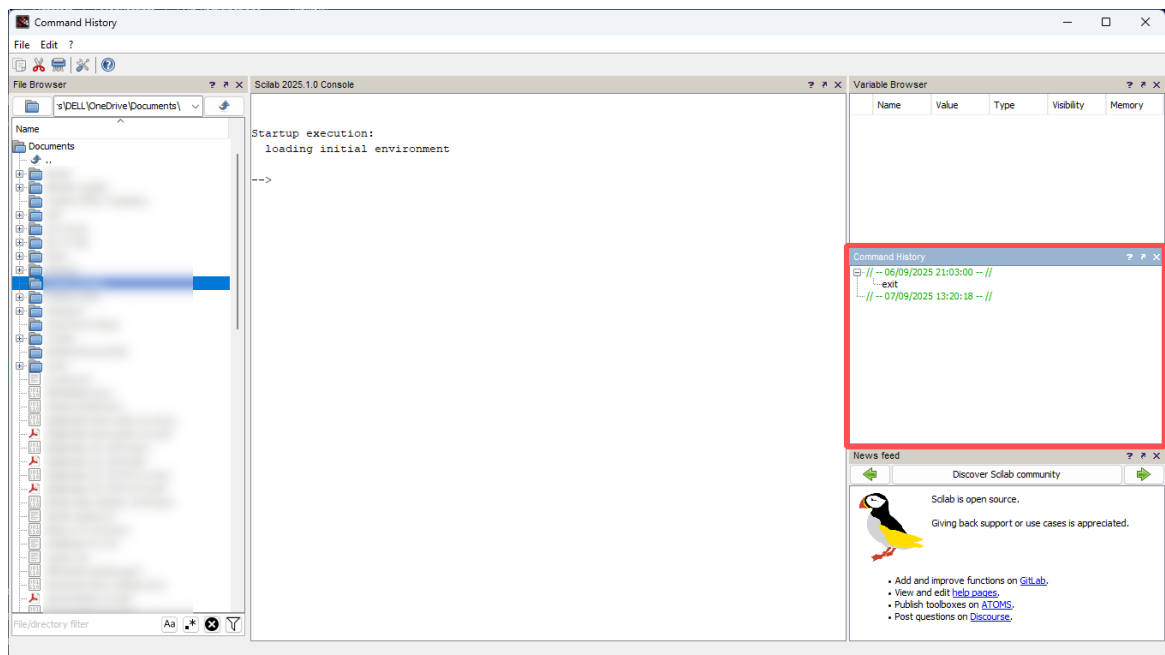
2. *Console window* – allows one to type and execute Scilab commands, as well as to view the output.



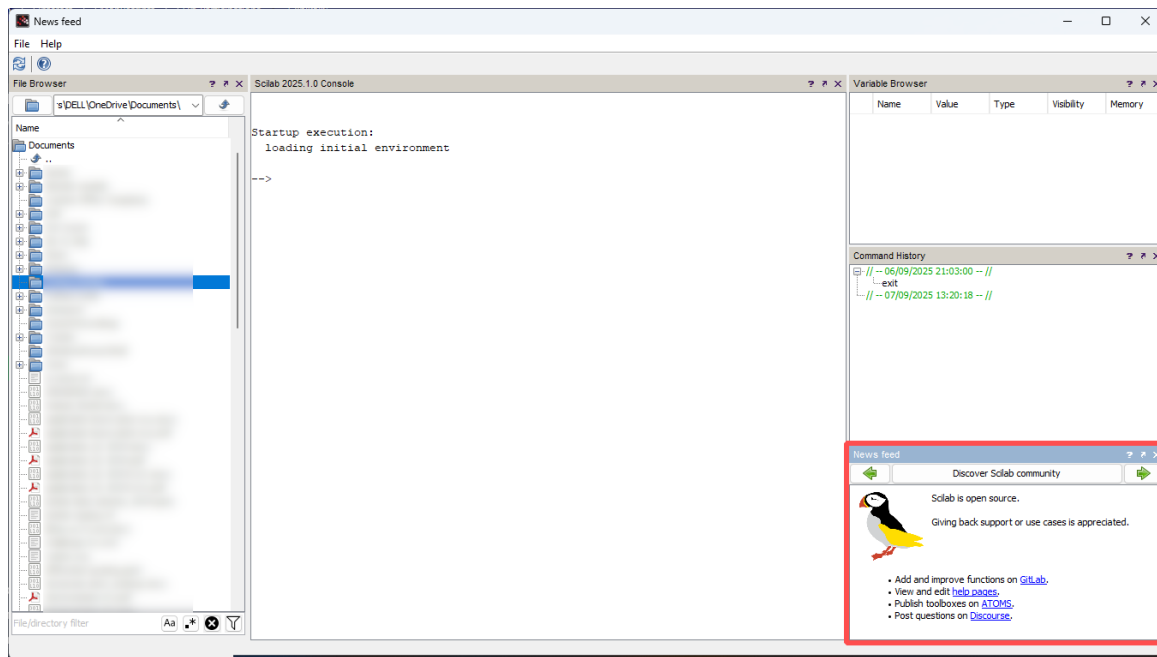
3. *Variable browser* – displays all the variable that have been created by Scilab and are currently active in memory locations.



4. *Command history* – display a chronological order of all the Scilab commands executed by the user, with the most recent command being at the top.



5. *News feed* – provides Scilab news and an interface to interact with the online Scilab community.



2.2 The Workspace

The Scilab workspace consists of the following components:

1. The console window

- Docked on the GUI by default.
- One can perform numerical calculations right from the console and view the output as soon as the computations are complete.
- One can find the current directory that one is working on by using the command `pwd` (which stands for *present working directory*) on the console window.
- The Scilab console presents a full-featured, interactive, command-line REPL (*read-evaluate-print-loop*) shell.

2. The variable browser

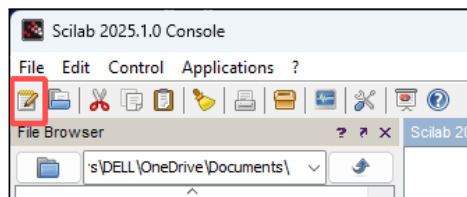
- Docked on the GUI by default.
- One can view the names of the variables, the value they store as well as the memory they are allocated.

3. The command history

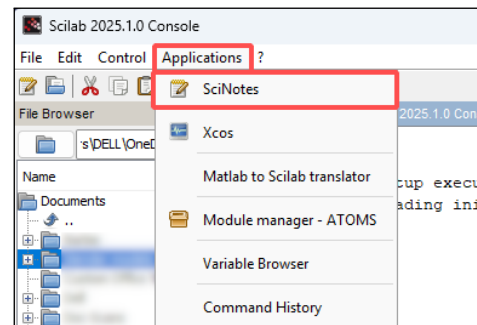
- Docked on the GUI by default.
- One can click on any of the earlier commands from the command history to re-run the same without having to type the command all over again.

4. **The SciNotes editor:** Running multiple lines of command is not possible on the console window. Also, the console window does not allow one to save a program such that it can be executed later. For these reasons, Scilab provides its own editor, known as SciNotes, that allow the user to write and save multi-line programs that can be executed at any point of time on any other PC with Scilab installed.

- Not docked on the GUI by default.
- To open SciNotes, one can either click on its icon on the taskbar (as shown in figure 5a), or one can use the main menu by clicking Applications → SciNotes (as shown in figure 5b).



(a) Opening SciNotes using the taskbar icon



(b) Opening SciNotes using the main menu

Figure 5: Two ways to open SciNotes

5. **The graphic window:** The graphic window is used for displaying any graphical output, such as the plot resulting from a numerical computation or a simulation result.

- Not docked on the GUI by default.
- A graphical window opens up automatically when making any plot.
- A graphical window may be opened by running any of the plot commands in Scilab either from the console window or by including it as a line in a Scilab program written using SciNotes.
- One can view a generic sample plot using a graphical window by simply typing `plot` on the console window, as illustrated in figure 6.

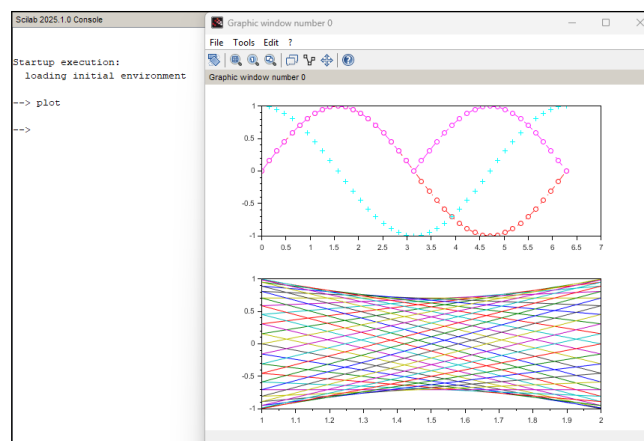


Figure 6: Generic sample plot on a graphical window

2.3 Documentation and help

Scilab provides plenty of resources in the form of documentation that can be easily accessed from the main menu by clicking ?. We are primarily interested in two of the sub-menu items, namely Scilab Help (which can be directly accessed by pressing the F1 key) and Scilab Demonstrations, as illustrated in figure 7.

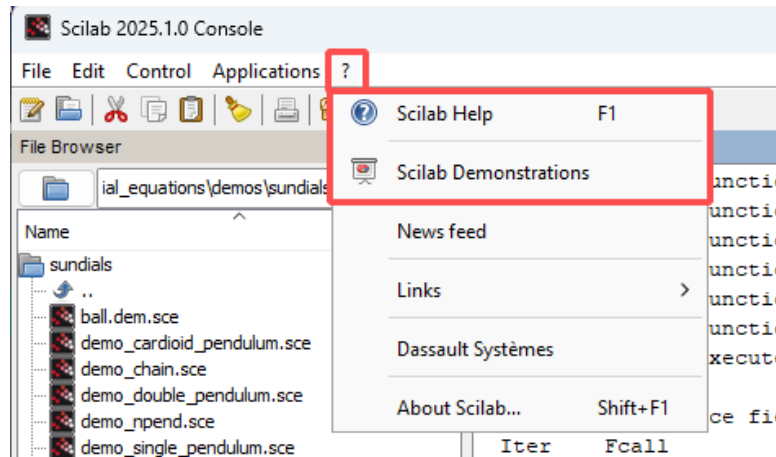


Figure 7: Documentation in Scilab

- **Scilab Help:** The Scilab Help Browser window provides detailed documentation on all the features and functions provided in various toolboxes that come bundled with Scilab.

One can either navigate to the exact topic using the hierarchical directory tree structure, or one may search for the specific topic of interest.

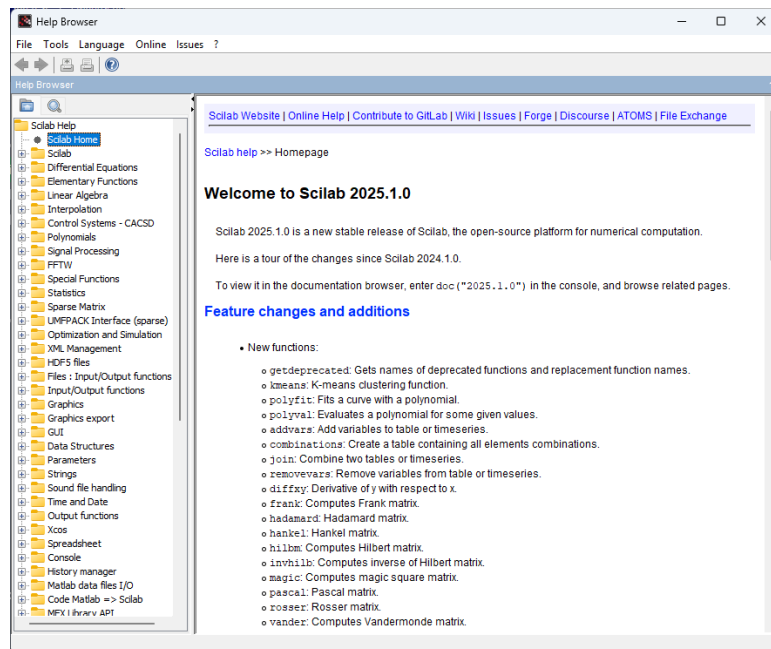


Figure 8: Scilab help browser window

- **Scilab Demonstrations:** Scilab provides a collection of demonstration scripts, which are available on the demonstrations menu. In figure 9, an example is displayed as to how one

can access a particular demonstration, in this case by clicking Simulation → ODE's → Lorenz equation from the demonstrations menu. This would open up a graphic window where the demonstration is performed. One can also view the underlying Scilab code by clicking on -- View Code -- on the main menu of the same graphic window.

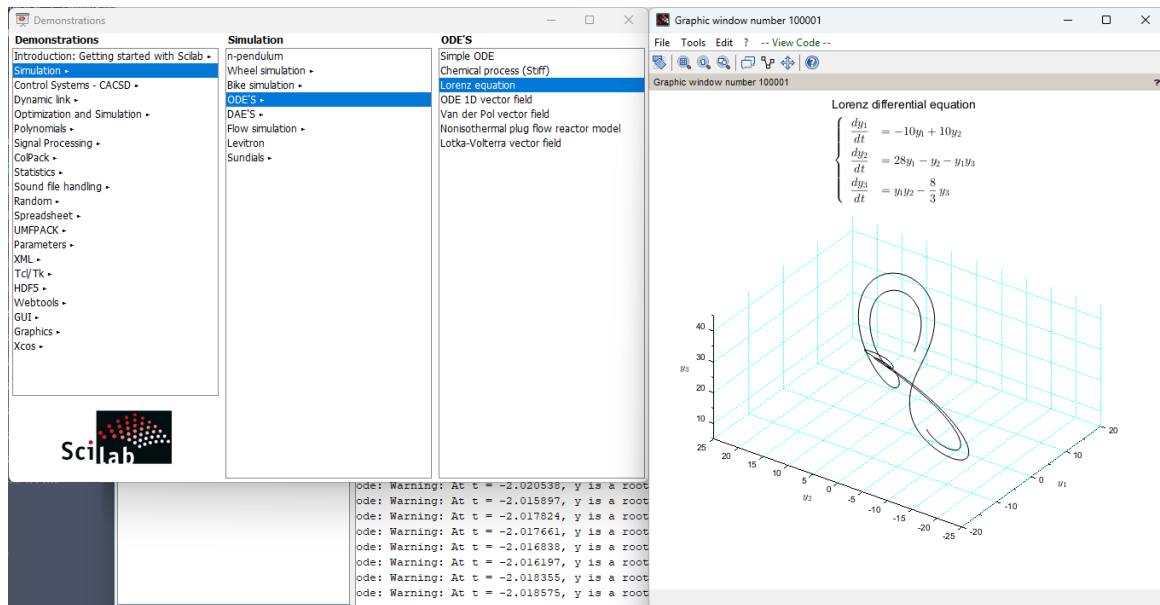


Figure 9: Scilab help browser window

3 Basic command execution

3.1 Arithmetic expressions

Scilab can be used as a simple calculator to perform numerical calculations. Some of such examples are given below.

```
--> 7 + 6  
ans =  
13.
```

```
--> sin(0.4156)  
ans =  
0.4037389
```

```
--> 9 / 17  
ans =  
0.5294118
```

```
--> acos(0.707)  
ans =  
0.7855492
```

```
--> 4 * 16  
ans =  
64.
```

```
--> exp(2.349)  
ans =  
10.475089
```

When Scilab evaluates an expression it stores the result in a variable so it can be used by subsequent statements; if the user does not explicitly assign a name, Scilab places the result in the built-in variable `ans`, whereas if the user provides an assignment the value is stored in that named variable instead, so explicit assignments are recommended when a value must be retained.

3.2 Variable assignments

Scilab allows the user to define variables and assign values to them so that they can be used later.

```
--> a = 3.4;  
--> b = 6.78;  
--> c = a + b  
c =  
10.18
```

```
--> w = exp(0.1);  
--> v = 2;  
--> u = w .^ v  
u =  
1.2214028
```

```
--> x = 13;  
--> z = x .* 2  
z =  
26.
```

```
--> pi = atan(1.0)*4;  
--> y = sin(pi/4)  
y =  
0.7071068
```

3.3 Managing output display

- Appending a semicolon (;) to an expression in Scilab suppresses the output echo – the expression is still evaluated and any assignment performed, but the resulting value is not printed in the console, which is useful for reducing clutter when running scripts or storing intermediate results.
- Multiple commands may be placed on a single line by separating them with either a semicolon (;) or a comma (,). Both delimiters allow successive statements to be evaluated and any assignments to occur, but a trailing or intervening semicolon suppresses the echo of intermediate results whereas a comma causes each result to be printed. So a semi-colon is useful for keeping the console uncluttered when running sequences of commands and a comma is convenient when the user wishes to inspect intermediate outputs.

```
--> a = 55; b = 16; c = 21;      --> a = 55, b = 16, c = 21,
--> disp([a b c])              a =
55.    16.    21.              55.
                                b =
                                16.
                                c =
                                21.
                                --> disp([a b c])
                                55.    16.    21.
```

- Semicolons and commas may be mixed on the same line: any expression terminated with a semicolon (;) is evaluated (and any assignment performed) without printing its value, whereas an expression terminated with a comma (,) is evaluated and its result is echoed to the console.

```
--> x = 150; y = 213, z = 574;
y =
213.
```

- To change the font on the console window or on SciNotes, click Edit → Preferences. Then click on Fonts on the left panel.

One can then uncheck the 'Use system font' option and proceed to select the desired font typeface.

Under 'Custom font' one can then select either Console or Scinotes and choose Desktop text font under 'Fonts to use'. Finally click on Apply and then on OK to change to the desired font.

- In order to clear the console window, one can either execute the `clc` (which stands for *clear console*) or simply press the F2 key.

4 Data types

4.1 Numerical data

Although modern machines commonly offer abundant memory and a 64-bit architecture, allocating the full 64 bits to every value (for example storing the small integer 0) is often wasteful; Scilab will by default manage data representation automatically, but this can lead to inefficiencies if the widest type is used indiscriminately. Consequently, the developer may either rely on Scilab's automatic choice or explicitly declare more appropriate data types (for example single precision or integer types) where suitable; used judiciously, explicit typing reduces memory consumption and can improve execution speed, while careless use of wide types increases both memory requirements and computational overhead.

Table 1: Numeric datatypes for integers along with precision

Data type	Meaning	Range
int8	Signed 8-bit integer	$[-2^7, 2^7 - 1]$
uint8	Unsigned 8-bit integer	$[0, 2^8 - 1]$
int16	Signed 16-bit integer	$[-2^{15}, 2^{15} - 1]$
uint16	Unsigned 16-bit integer	$[0, 2^{16} - 1]$
int32	Signed 32-bit integer	$[-2^{31}, 2^{31} - 1]$
uint32	Unsigned 32-bit integer	$[0, 2^{32} - 1]$
int64	Signed 64-bit integer	$[-2^{63}, 2^{63} - 1]$
uint64	Unsigned 64-bit integer	$[0, 2^{64} - 1]$

The limits of the data types given in table 1 can be tested as follows:

```
--> int16(2^15)          --> uint16(2^16-1)
ans = [int16]            ans = [uint16]
-32768                   65535

--> int16(2^15-1)        --> uint16(2^16)
ans = [int16]            ans = [uint16]
32767                    0
```

A floating-point number is represented in the following normalized form:

$$(-1)^S M \times B^E$$

where S is the *sign*, M is the *mantissa*, B is the *base* and E is the *exponent*. Each of these components occupies specific bit-fields in memory, and the total number of bits used to represent the

value is referred to as its precision. Scilab supports several floating-point precisions (listed in table 2), and understanding how sign, mantissa and exponent are allocated helps explain rounding behaviour, range and storage cost for numerical values.

Table 2: Precisions available for floating point numbers

Precision	Bits occupied	Bytes allocated
Single	32	4
Double	64	8
Extended double	80	10
Quadruple	128	16

On a 64-bit system running Scilab, real (floating-point) variables are typically represented using 64 bits in the IEEE 754 double-precision format – hence the data type name `double` – which provides a wide dynamic range and sufficient precision for most scientific computations. This format includes the following:

- 1 bit to store the sign S , i.e. $2^1 = 2$ possible values (0 for +, 1 for –).
- 52 bits to store the mantissa M , i.e. $2^{52} = 4.5 \times 10^{15}$ possible values.
- 11 bits to store the exponent E , i.e. $2^{11} = 2048$ possible values.

Formatted display of numerical data: Scilab provides two complementary ways to control how numeric results appear: a *global display format* that governs the default printing of numbers at the prompt, and *explicit format strings* for precise control when producing output to the console or to files. Use the global format for convenient, interactive inspection and use the explicit format functions when exact field widths, number of digits or layout are required.

1. *Global display format:* The `format` function sets the default way Scilab prints decimal numbers at the prompt and in functions such as `disp`. It supports a *mode* (for example a variable/default ‘v’ mode or an exponential ‘e’ mode) and a *width* (the number of output characters used, all included: sign of the mantissa, its digits, decimal separator, exponent symbol, sign and digits of the exponent).

The call forms include `format(mode)`, `format(width)`, and `format(mode, width)`, and `format()` returns the current settings. Changing the global format is convenient for interactive work when one wants more or fewer significant digits without rewriting print statements.

```
--> pi = 22/7;

--> disp(pi)
3.1428571

--> format("v",6)
```

```
--> disp(pi)
3.143

--> format("e",10)

--> disp(pi)
3.143D+00
```

2. *Explicit format strings*: For formatted output (tables, reports, iteration logs, files), Scilab implements C-style conversion specifiers via `mprintf` (console), `mfprintf` (file) and `msprintf` (return a string). These functions accept format strings containing conversion specifiers such as `%f`, `%e`, `%g`, and allow specifying field width, precision and numbered placeholders.

4.2 Boolean data

Boolean data is assigned to variables using either `%T` (for *true*) or `%F` (for *false*).

```
--> a = %T
a =
T
```

Boolean variables must be operated using boolean operators which are the following:

- NOT – represented by the symbolic operator `~`

```
--> b = ~a
b =
F
```

- AND – represented by the symbolic operator `&`

```
--> a & b
ans =
F
```

- OR – represented by the symbolic operator `|`

```
--> a | b
ans =
T
```


4.3 Strings

Alphabets along with special characters are treated as string data types. String variable and values can be concatenated using the + operator.

```
--> s1 = "Hello";

--> s2 = "world";

--> s = s1 + s2
s =
    "Helloworld"

--> s = s1 + ", " + s2 + "!"
s =
    "Hello, world!"
```

The built-in function `string()` converts a given value to string data type. As shown in the following example, the numerical value 2 is converted to a string data type when the `string()` function is applied to it.

```
--> a = 2
a =
    2.

--> s = string(a)
s =
    "2"
```

One can now concatenate the string format of “2” with another number using the + operator.

```
--> t = string(5)
t =
    "5"

--> s + t
ans =
    "25"
```

4.4 Complex numbers

Scilab supports complex numbers as a separate numeric type, using the imaginary unit %i (also available as the shorthand 1i or 1j) so that expressions such as $2 + 3\%i$ or $2 + 3i$ represent the complex number $2 + 3i$.

```
--> %i                                --> 2 + 3*%i
%i =                                  ans =
0. + i                                2. + 3.i

--> 1i                                --> 2 + 3i
ans =                                 ans =
0. + i                                2. + 3.i
```

Complex numbers can also be created from separate real and imaginary parts with the constructor `complex(a, b)`, which is particularly convenient for arrays.

```
--> z = complex(4, -5)
z =
4. - 5.i
```

The real and imaginary parts of a complex number can be extracted using the `real()` and `imag()` functions respectively.

```
--> real(z)                            --> imag(z)
ans =                                  ans =
4.                                    -5.
```

5 Operators and expressions

Having looked at the data types Scilab works with, we now look at the operators that combine them into expressions. Scilab's operator set will look familiar to anyone who has used a calculator, but a few of its precedence and associativity rules are genuinely unusual.

5.1 Arithmetic operators

Operator	Operation	Example	Result
+	Addition	5 + 3	8.
-	Subtraction	5 - 3	2.
*	Multiplication	5 * 3	15.
/	Right division	4 / 2	2.
\	Left division	4 \ 2	0.5
^	Exponentiation	2 ^ 5	32.

- In case of scalars, when we write $a \setminus b$, it means when “b is divided by a”, i.e. $a \setminus b$ is the same as b / a . However, in the case of arrays and matrices (e.g. solving a system of linear equations), $A \setminus b$ is not the same as b / A .
- In section 3.2, we had written expressions such as $z = x .* 2$ and $u = w .^ v$. In case of scalars, writing $.*$ is identical to writing $*$, and so on. The difference arises later once we start operating on vectors and matrices. Then the dot (.) preceding the operator implies element-by-element operations between vectors and/or matrices.

5.2 Relational operators

Operator	Operation	Example	Result
==	Equal to	7 == 5	F
~= or <>	Not equal to	8 ~= 9 or 8 <> 9	T
<	Less than	9 < 9	F
<=	Less than or equal to	9 <= 9	T
>	Greater than	6 > 4	T
>=	Greater than or equal to	6 >= 4	T

- The result of a relational operator is always boolean, i.e. F implies ‘false’ and T implies ‘true’.

- Both `~=` and `<>` are two versions of the same operator. Scilab keeps both for historical reasons and it does not matter which is used, though it is good practice to pick one and use it consistently within a script.

5.3 Logical operators

In section 4.2, three logical operators `&` (AND), `|` (OR) and `~` (NOT) were introduced, which operate on boolean data. We can use them to build compound conditions out of relational expressions.

```
--> temp = 305;
--> pres = 1.2;
--> (temp > 273) & (pres <= 1.5)
ans =
T
```

```
--> temp = 305;
--> pres = 1.2;
--> (temp < 273) | (pres <= 1.5)
ans =
T
```

5.4 Operator precedence and associativity

Scilab's own documentation does not print a single consolidated precedence table anywhere. These rules have to be pieced together from the documentation of individual operators, or determined empirically. The following ordering (highest precedence first) is sufficient for every expression one is likely to write.

Precedence	Operation	Operators
1 (highest)	Parentheses	()
2	Transpose	' or .'
	Power	^ or .^
3	Multiplication	* or .*
	Division	/ or ./ and \ or .\
4	Addition	+ or .+
	Subtraction	- or .-
5	Logical NOT	~
6	Relational operations	==, ~=, <>, <, <=, >, >=
7	Logical AND	&
8 (lowest)	Logical OR	

Note: When in doubt about how an expression will be interpreted by Scilab, parenthesise it explicitly. Even though precedence rules exist so that one rarely has to use parentheses, they do not mean one should never use them for clarity.

6 Code structure and readability

A script that Scilab can execute correctly is not the same thing as a script that a human being can read correctly. Here we discuss the conventions that regarding how to name things sensibly and how to document what a piece of code is doing and why so that it does not become unreadable.

6.1 Variable naming conventions

Scilab's rules for a legal variable name are as follows:

- i. A variable name must begin with a letter or an underscore (_).

Examples:

- Temperature and `_density` are allowed.
- `5freq` and `21_dydx` are not allowed.

- ii. A variable name may thereafter contain letters, digits and underscores.

- `xa`, `force1` and `field_` are allowed.
- `const_@`, `vol 5` are not allowed.

- iii. A variable name is case-sensitive.

- `vector_1` is different from `Vector_1`.

Apart from the above rules, it is advisable to follow the conventions given below so as to ensure that one's code is readable.

1. Prefer descriptive names for anything that persists beyond a few lines. Single-letter names such as `i`, `j`, `x` are fine for loop counters and short-lived scratch values, but a variable that stores a physical quantity should explicitly say so., say, a radon concentration or a decay constant
 - E.g. the variable name `radon_conc` is more appropriate than `rc` for storing the value of radon concentration.
2. Choose a particular variable naming convention and adhere to it. Some of the commonly used conventions are given below.
 - *Camel case*: The first word starts with a lowercase letter, and every following word begins with a capital letter with no spaces or symbols in between. E.g. `momentOfInertia`, `chargeDensity` and `workFunction`.
 - *Pascal case*: Similar to camel case, but the very first word also starts with a capital letter. E.g. `MomentOfInertia`, `ChargeDensity` and `WorkFunction`.
 - *Snake case*: All letters are lowercase, and words are separated by an underscore (_). E.g. `moment_of_inertia`, `charge_density` and `work_function`.
3. Avoid naming variable after built-in function names. Even though Scilab does not prevent one from doing so, it would hide the built-in function for the rest of the session.

- E.g. naming a variable as `sum` or `mean` would hide the `sum()` or `mean()` functions from being used.
4. Avoid naming variables after protected % constants, even though both would work. Doing so might lead to code that would be confusing later on. A list of all protected constants in Scilab is given in the table below.
- E.g. naming a variable as `pi` is allowed and the protected `%pi` constant is still accessible. But it is still poor practice to reuse a name this close to a well-known constant purely because it invites confusion when a script is read quickly.

Constant	Meaning
<code>%pi</code>	Ratio of a circle's circumference to its diameter
<code>%e</code>	Euler's constant, which is the base of natural logarithms
<code>%eps</code>	Floating-point relative accuracy / machine precision
<code>%i</code>	Imaginary unit which is equivalent to $\sqrt{-1}$
<code>%inf</code>	Infinity, resulting from operations, such as $1/0$
<code>%nan</code>	Not-a-Number, representing an undefined or unrepresentable value
<code>%t</code> or <code>%T</code>	Boolean value for true state
<code>%f</code> or <code>%F</code>	Boolean value for false state
<code>%s</code>	Special variable used to define continuous-time polynomials
<code>%z</code>	Special variable used to define discrete-time polynomials

6.2 Using comments

Inserting comments within Scilab code is important because it improves the readability of the code and integrates the documentation within the code itself. Even though using comments is not mandatory, they are immensely useful because they ensure that the purpose of the code is properly understandable.

Scilab recognises two comment styles.

1. Single line comments
2. Block comments

6.2.1 Single line comments

These comments begin with `//` from the point where the comment is required to begin. Scilab ignores everything from `//` to the end of the line.

Example:

```
// This is a full-line comment  
x = 5; // This is an inline comment
```

6.2.2 Block comments

These comments begin with a `/*` and end with a `*/`, and Scilab ignores everything in-between. Such comments can, therefore, span multiple lines without requiring `//` on each line.

Example:

```
/* This is a comment block  
that spans across  
multiple lines */
```

7 Scripts, saving and execution workflow

In section 2.2, we introduced the SciNotes editor that allows us to write Scilab scripts and execute them at any other time on any other PC. Here we return to SciNotes to understand the workflow involved in the execution of a Scilab script.

7.1 Script file formats

A file written in SciNotes is saved in one of two conventional formats, distinguished by extension:

1. The `.sce` format is a script file, which may contain any mixture of executable Scilab statements and function definitions.
2. The `.sci` format is a file intended to hold only function definitions. Running a `.sci` file loads the functions it contains into the Scilab workspace, ready to be called, but does not execute any statement by itself, since by convention it should not contain any bare statements to execute.

7.2 Creating a script

To create a new script, we can do either of the following:

1. Click on the New button on the SciNotes toolbar.
2. Click on File > New from the main menu in SciNotes.

These are illustrated in figure 10.

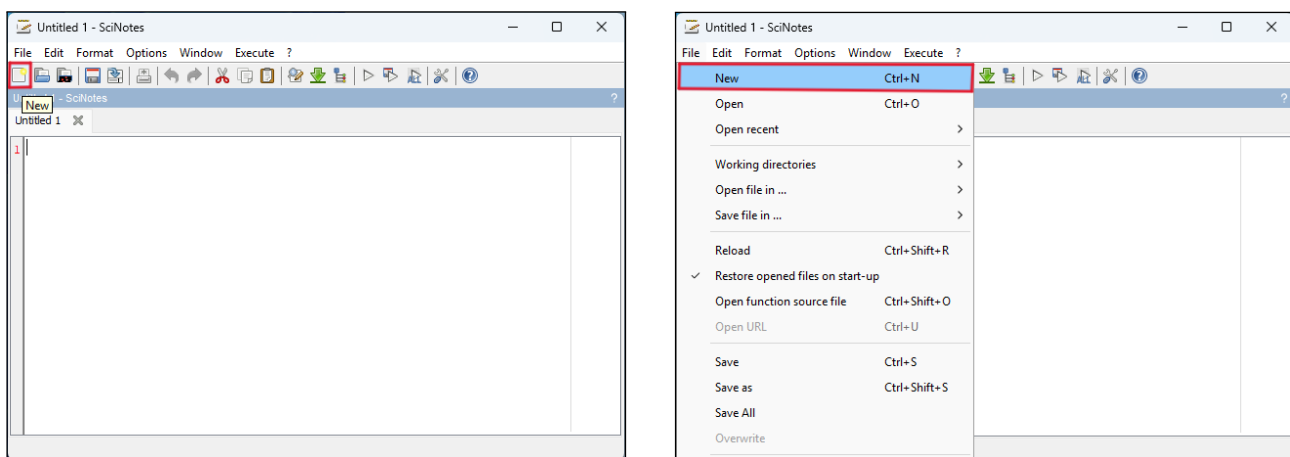


Figure 10: Two ways to create a new script in SciNotes

7.3 Saving a script

Once a Scilab script has been written, it can be saved as follows:

- I. Clicking on the Save button on the toolbar or click on File > Save from the main menu in Scilab.
- II. Enter a file name, appended with either .sce or .sci in the file save dialog and then click the Save button.

These steps are illustrated in figure 11.

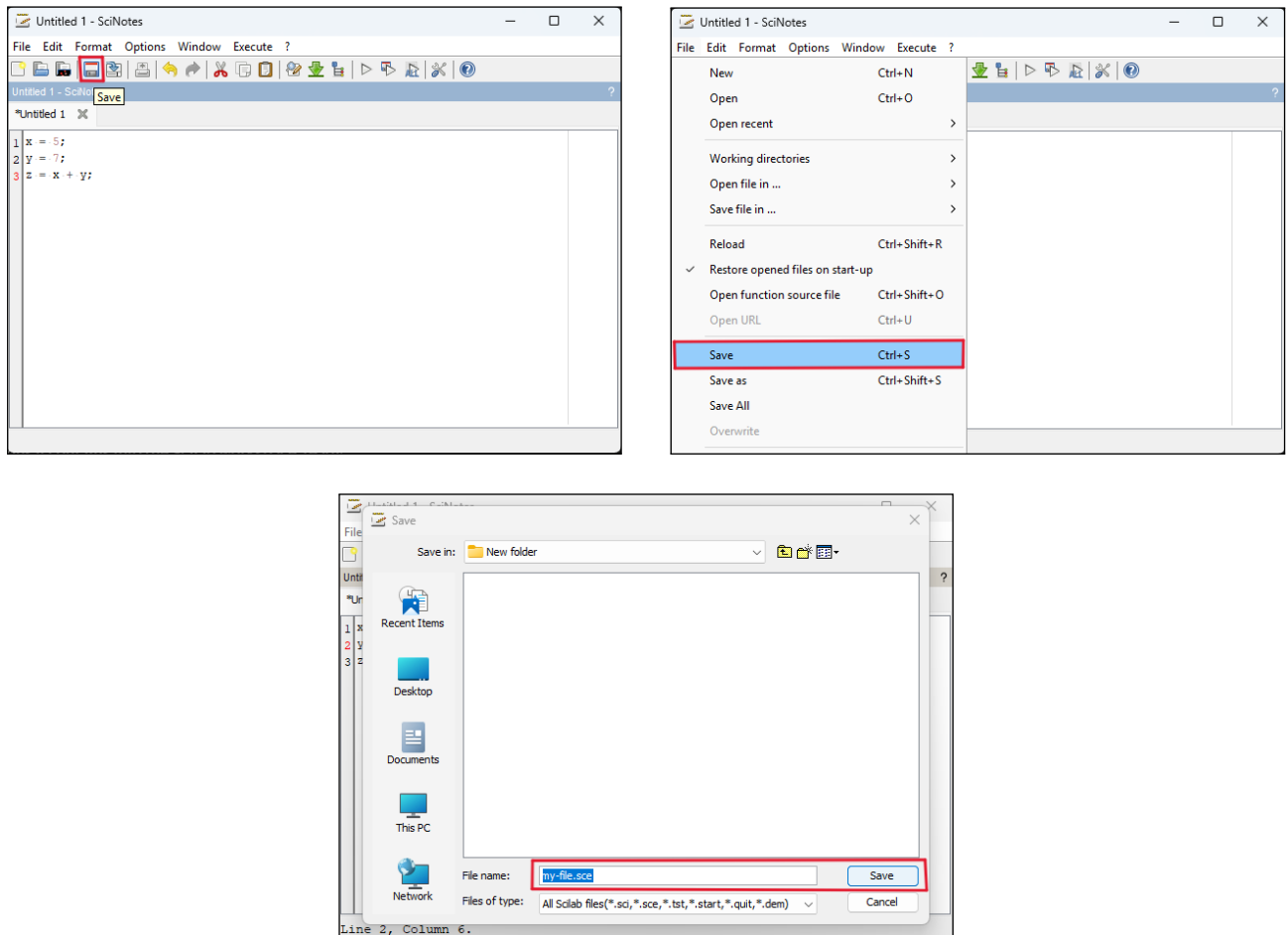


Figure 11: Saving a script in SciNotes

7.4 Executing a script

To execute a script in Scilab, we can click any of the following buttons:

1. The Execute button which would execute the current file if the changes are already saved.
2. The Save and execute button which first saves the current file and then executes the same.
3. The Save and execute all button which first saves all the files open as tabs and then executes all of them.

The locations of the above buttons in the toolbar are shown in figure 12.

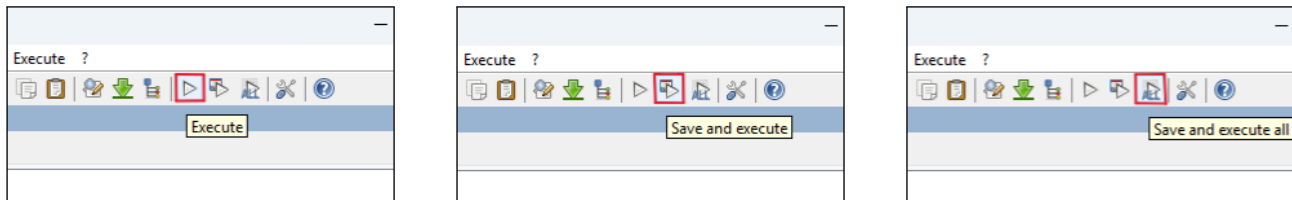


Figure 12: Executing a script in SciNotes

An alternate and more flexible way to execute a script in Scilab is by using the `exec()` function which takes the path to a file and executes its contents sequentially. For instance, to execute a Scilab script named `sample-script.sce` which is saved on the Desktop, we can run the following command in the Scilab console:

```
exec('C:\Users\Desktop\sample-script.sce')
```

The above command would execute the script by echoing all the lines in the script. To suppress this echo, we can run the `exec()` command with an additional `-1` argument:

```
exec('C:\Users\Desktop\sample-script.sce', -1)
```

The practical advantage of the `exec()` over the SciNotes menu is that it can be typed directly at the console, chained with other commands, or embedded inside another script. For instance, one script that loads a shared set of function definitions from a `.sci` file with `exec()` before getting on with its own calculation.